

# 大题专项——练习题解析

## 板块一：关系理论·基础

### 第1题

答案：

- (1) L类：B；R类：F；LR类：A、C、D、E
- (2) 候选码：BE
- (3)  $(BD)^+ = ABCDEF$

解析：这题考的是候选码求解的完整流程：L/R/LR 划分 → 必含属性入手 → 闭包验证 → 求某属性组闭包。

(1) L/R/LR 划分

按 F 中每个属性出现位置分类：

- A：AC→D 左侧出现、E→A 右侧出现 → LR
- B：B→C、B→F 左侧出现，右侧从未出现 → L
- C：B→C 右侧出现、AC→D 左侧出现 → LR
- D：AC→D 左侧出现、D→E 右侧出现 → LR
- E：D→E 右侧出现、E→A 左侧出现 → LR
- F：B→F 右侧出现，左侧从未出现 → R

L 类：B；R 类：F；LR 类：A、C、D、E

(2) 求所有候选码

L 类 B 必在候选码中。逐个试 LR 类与 B 组合求闭包：

- $B^+ = \{B, C, F\}$  (用 B→C、B→F) → 不全
- $BD^+ = \{B, D\}$  起步；B→C 加 C；B→F 加 F；D→E 加 E；E→A 加 A；AC→D 已含 D →  $BD^+ = \{A, B, C, D, E, F\}$  → BD 是超码
- 验极小：BD 去掉 D 后剩 B， $B^+ = BCF$  不全 → BD 是极小超码 → BD 是候选码
- $BE^+ = \{B, E\}$  起步；B→C、B→F 加 C、F；E→A 加 A；AC→D 加 D；D→E 已含 E →  $BE^+ = \{A, B, C, D, E, F\}$  → BE 是超码
- 验极小：BE 去掉 E 后剩 B， $B^+ = BCF$  不全 → BE 是极小超码 → BE 是候选码

候选码 = {BE, BD} (注意 BDE 是 BE 的超集，不是独立的候选码——候选码集合只能由极小超码组成)

(3) 求  $(BD)^+$  闭包

逐步推导：

- 初始：{B, D}
- 用 B→C，加 C → {B, C, D}
- 用 B→F，加 F → {B, C, D, F}
- 用 D→E，加 E → {B, C, D, E, F}
- 用 E→A，加 A → {A, B, C, D, E, F}
- AC→D 已含 D，无新元素
- 终止： $(BD)^+ = ABCDEF$

双重验证： $(BD)^+$  包含 U 中所有 6 个属性 → BD 是超码；又 BD 去掉 D 后剩 B， $B^+ = BCF$  不全 → BD 是极小超码 → 确认 BD 是候选码 (与上面一致)。

金句：L 类是“必含种子”——候选码必含 L 类，但 LR 类是“待定补集”——逐个试到能推全集为止。

闭包计算要"用过的 FD 可重复扫，直到无新元素"。候选码集合只能由极小超码组成，不能含超集。

## 第 2 题

答案：

- (1) 候选码：AB；主属性：A、B；非主属性：C、D、E
- (2) 1NF【正确】、2NF【正确】、3NF【错误】、BCNF【错误】；最高范式为 2NF

解析：这题考的是范式级别判断的递进流程。

(1) 求候选码

L/R/LR 划分（先看 F 中位置）：

- A：AB→C 左侧 → L
- B：AB→C 左侧 → L
- C：AB→C 右侧、C→D 左侧、C→E 左侧 → LR
- D：C→D 右侧 → R
- E：C→E 右侧 → R

L 类 A、B 必在码中。试 AB：

- $AB^+ = \{A, B, C, D, E\}$  (AB→C 加 C；C→D 加 D；C→E 加 E) →  $AB^+ = U$  【正确】

候选码：AB。主属性 A、B；非主属性 C、D、E (C 不在 AB 中，也是非主属性)。

(2) 逐级判断

1NF【正确】：所有属性 A、B、C、D、E 都是原子项（题目未给出复合属性），满足 1NF。

2NF【正确】：2NF 要求每个非主属性完全函数依赖于候选码 AB。

- C：依赖为 AB→C，左侧 AB 是码（非真子集）→ 完全依赖 【正确】
- D：依赖为 C→D，左侧 C 是非主属性（不是码，更不是码的真子集）→ 不构成"部分依赖"（部分依赖的定义是"左侧是码的真子集"）【正确】
- E：依赖为 C→E，同理 【正确】

非主属性 C、D、E 没有对码 AB 的部分函数依赖 → 2NF【正确】。

3NF【错误】：3NF 要求不存在非主属性对码的传递函数依赖。

- 路径：AB→C（码 AB 决定非主属性 C），C→D（非主属性 C 决定非主属性 D）
- D 通过非主属性 C 作为桥梁间接被 AB 决定 → D 对 AB 存在传递函数依赖
- 同理 E 也有传递函数依赖
- 3NF【错误】。

BCNF【错误】：3NF 已经不满足，BCNF 必不满足。检查 BCNF 条件（每个决定因素都含码）：

- C→D 左侧 C 不是码 → 违反 BCNF
- C→E 左侧 C 不是码 → 违反 BCNF
- BCNF【错误】。

最高范式为 2NF。

金句：3NF vs 2NF 关键看"传递"——非主属性作为桥梁决定另一个非主属性，就是传递函数依赖。BCNF vs 3NF 关键看"所有依赖"——BCNF 要求每一个决定因素（含非主属性决定的）左侧都含码。

## 第 3 题

答案：Fmin = {A→B, D→A, AB→C, AB→D, BD→E}

解析：这题考的是求最小函数依赖集的三步标准流程。

Step 1：拆分右侧

F 中右侧为多属性的依赖只有 AB→CDE，按属性拆分为单属性：



•  $AB \rightarrow CDE \rightarrow AB \rightarrow C, AB \rightarrow D, AB \rightarrow E$

$F1 = \{A \rightarrow B, D \rightarrow A, AB \rightarrow C, AB \rightarrow D, AB \rightarrow E, BD \rightarrow E\}$

Step 2：去除自身求闭包

对  $F1$  中每条 FD  $X \rightarrow Y$ ，尝试在  $F1 - \{X \rightarrow Y\}$  下求  $X^+$ ，若已包含  $Y$  则删除  $X \rightarrow Y$ 。

| FD                 | 在 $F1 - \{X \rightarrow Y\}$ 下求 $X^+$                                                                                                                            | 闭包                  | 含 $Y$ ? | 处理 |
|--------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------|---------|----|
| $A \rightarrow B$  | $(A)^+$ 仅有 $\{A\}$ ( $D \rightarrow A, AB \rightarrow ?$ 、 $B \rightarrow D \rightarrow E$ 都需要 $D$ 或 $B$ )                                                       | $\{A\}$             | 否       | 保留 |
| $D \rightarrow A$  | $(D)^+$ 仅有 $\{D\}$ ( $A \rightarrow B$ 需 $A$ ， $AB \rightarrow ?$ 需 $A$ ， $BD \rightarrow E$ 需 $B$ )                                                             | $\{D\}$             | 否       | 保留 |
| $AB \rightarrow C$ | $(AB)^+$ 含 $D$ 、 $E$ 、 $B$ ( $A \rightarrow B$ 已含 $B$ ； $AB \rightarrow D$ 加 $D$ ； $AB \rightarrow E$ 加 $E$ ； $BD \rightarrow E$ 需 $D$ ，加 $E$ 已含 )               | $\{A, B, D, E\}$    | 否       | 保留 |
| $AB \rightarrow D$ | $(AB)^+$ 含 $C$ 、 $E$ ( $A \rightarrow B$ 已含 $B$ ； $AB \rightarrow C$ 加 $C$ ； $AB \rightarrow E$ 加 $E$ ； $BD \rightarrow E$ 需 $D$ )                               | $\{A, B, C, E\}$    | 否       | 保留 |
| $AB \rightarrow E$ | $(AB)^+$ 含 $C$ 、 $D$ ( $A \rightarrow B$ 已含 $B$ ； $AB \rightarrow C$ 加 $C$ ； $AB \rightarrow D$ 加 $D$ ； $BD \rightarrow E$ 需 $B$ 和 $D$ ，加 $E$ ) $\rightarrow$ 全集 | $\{A, B, C, D, E\}$ | 是       | 删除 |
| $BD \rightarrow E$ | $(BD)^+$ 仅有 $\{B, D\}$ ( $A \rightarrow B$ 需 $A$ ， $AB \rightarrow ?$ 需 $A$ )                                                                                    | $\{B, D\}$          | 否       | 保留 |

$F2 = \{A \rightarrow B, D \rightarrow A, AB \rightarrow C, AB \rightarrow D, BD \rightarrow E\}$

Step 3：左侧最小化

对  $F2$  中每条 FD  $X \rightarrow Y$ ， $X$  多属性时尝试逐个去掉  $X$  中的属性  $Z$ ，看在  $F2$  下  $Z$  是否能由  $X - \{Z\}$  推出 ( 进而  $X$  仍能推  $Y$  )。

| FD                 | 候选最小左侧                                                        | 检验      | 保留?                |
|--------------------|---------------------------------------------------------------|---------|--------------------|
| $A \rightarrow B$  | $A$ 单个                                                        | 无可去     | $A \rightarrow B$  |
| $D \rightarrow A$  | $D$ 单个                                                        | 无可去     | $D \rightarrow A$  |
| $AB \rightarrow C$ | 去 $A$ : $(B)^+ = \{B\}$ 不含 $C$                                | $A$ 不可去 | $AB \rightarrow C$ |
| $AB \rightarrow C$ | 去 $B$ : $(A)^+ = \{A, B\}$ ( $A \rightarrow B$ 加 $B$ ) 不含 $C$ | $B$ 不可去 | $AB \rightarrow C$ |
| $AB \rightarrow D$ | 去 $A$ : $(B)^+ = \{B\}$ 不含 $D$                                | $A$ 不可去 | $AB \rightarrow D$ |
| $AB \rightarrow D$ | 去 $B$ : $(A)^+ = \{A, B\}$ 不含 $D$                             | $B$ 不可去 | $AB \rightarrow D$ |



|      |                                                                           |       |      |
|------|---------------------------------------------------------------------------|-------|------|
| BD→E | 去 B : (D)+ = {A, B, C, D} ( D→A 加 A ; A→B 加 B ; AB→C 加 C ; AB→D 已含 ) 不含 E | B 不可去 | BD→E |
| BD→E | 去 D : (B)+ = {B} 不含 E                                                     | D 不可去 | BD→E |

$F_{min} = \{A \rightarrow B, D \rightarrow A, AB \rightarrow C, AB \rightarrow D, BD \rightarrow E\}$

双重验证：

- 拆分右侧、去除冗余、左侧最小化三步均已做 【正确】
- $F_{min}$  中每条 FD 左侧都是单个或不可约 【正确】
- $F_{min}$  中不存在能从其他依赖推出的依赖 ( Step 2 已验证 ) 【正确】
- $F_{min}$  与  $F$  等价—— $F$  中  $AB \rightarrow CDE$  在  $F_{min}$  下  $(AB)^+ = \{A, B, D, E\} + C ( AB \rightarrow C ) = ABCDE$  【正确】， $BD \rightarrow E$  在  $F_{min}$  下  $(BD)^+ = \{B, D, A, C, E\} ( BD \rightarrow A, A \rightarrow B \text{ 已含 } B, AB \rightarrow C, AB \rightarrow D, BD \rightarrow E \text{ 加 } E ) = ABCDE$  【正确】

金句：求  $F_{min}$  三步是“拆右、去冗、缩左”——拆右看是否多属性、去冗看自身闭包、缩左看单属性能否由其他推出。最小集不唯一（可能有多个等价  $F_{min}$ ），但每一步都必须做完。

## 板块二：关系理论·分解

### 第 4 题

答案：

- ( 1 ) 候选码：AB
- ( 2 )  $R_1(ABC)$ 、 $R_2(AD)$ 、 $R_3(DE)$ 、 $R_4(CF)$
- ( 3 ) 全部满足 3NF
- ( 4 ) 保持依赖

解析：这题考的是分解为 3NF 的标准流程（“ $F_{min}$  拆无合同 + 候选码单飞”）。

(1) 求候选码

$F_{min} = \{AB \rightarrow C, A \rightarrow D, D \rightarrow E, C \rightarrow F\}$

- A :  $AB \rightarrow C$  左侧、 $A \rightarrow D$  左侧，无右侧 → L
- B :  $AB \rightarrow C$  左侧，无右侧 → L
- C :  $AB \rightarrow C$  右侧、 $C \rightarrow F$  左侧 → LR
- D :  $A \rightarrow D$  右侧、 $D \rightarrow E$  左侧 → LR
- E :  $D \rightarrow E$  右侧 → R
- F :  $C \rightarrow F$  右侧 → R

L 类 A、B 必在候选码中。

- $AB^+ = \{A, B, C, D, E, F\} ( AB \rightarrow C \text{ 加 } C ; A \rightarrow D \text{ 加 } D ; D \rightarrow E \text{ 加 } E ; C \rightarrow F \text{ 加 } F ) \rightarrow AB^+ = U$  【正确】，AB 是候选码

候选码：AB。主属性 A、B；非主属性 C、D、E、F。

(2) 分解为 3NF

按“ $F_{min}$  拆无合同 + 候选码单飞”流程：

- Step 1 :  $F_{min} = \{AB \rightarrow C, A \rightarrow D, D \rightarrow E, C \rightarrow F\}$  ( 题目已给 )
- Step 2 :  $F_{min}$  中所有属性都出现在某个依赖中 ( 无未出现属性需要单独分组 )
- Step 3 : 同左部合并——
- $AB \rightarrow C \rightarrow R_1(ABC)$
- $A \rightarrow D \rightarrow R_2(AD)$



- $D \rightarrow E \rightarrow R_3(DE)$
  - $C \rightarrow F \rightarrow R_4(CF)$
  - Step 4：候选码 AB 已完整出现在  $R_1(ABC)$  中 ( A 和 B 都在 ABC 内 )  $\rightarrow$  无需再单独加候选码关系
- 分解结果： $R = \{R_1(ABC), R_2(AD), R_3(DE), R_4(CF)\}$

### (3) 验证 3NF

| 关系         | F 投影               | 候选码 | 非主属性 | 3NF 结论                                   |
|------------|--------------------|-----|------|------------------------------------------|
| $R_1(ABC)$ | $AB \rightarrow C$ | AB  | C    | 满足 ( 只有 $AB \rightarrow C$ ，左侧含码 ) 【正确】  |
| $R_2(AD)$  | $A \rightarrow D$  | A   | D    | 满足 ( A 是码， $A \rightarrow D$ 左侧含码 ) 【正确】 |
| $R_3(DE)$  | $D \rightarrow E$  | D   | E    | 满足 ( D 是码， $D \rightarrow E$ 左侧含码 ) 【正确】 |
| $R_4(CF)$  | $C \rightarrow F$  | C   | F    | 满足 ( C 是码， $C \rightarrow F$ 左侧含码 ) 【正确】 |

全部满足 3NF。

### (4) 保持依赖验证

| FD                 | 检查位置               | 是否保持 |
|--------------------|--------------------|------|
| $AB \rightarrow C$ | $R_1(ABC)$ 含 A、B、C | 是    |
| $A \rightarrow D$  | $R_2(AD)$ 含 A、D    | 是    |
| $D \rightarrow E$  | $R_3(DE)$ 含 D、E    | 是    |
| $C \rightarrow F$  | $R_4(CF)$ 含 C、F    | 是    |

分解保持函数依赖。

无损连接验证：分解 R 中  $R_1(ABC)$  包含候选码 AB ( A 和 B 都在 ABC 中 )  $\rightarrow$  分解具有无损连接性 【正确】 ( 定理：分解 R 中只要有一个分解关系包含候选码，则分解具有无损连接性 )。

金句：3NF 分解套路是“Fmin 拆无合同、候选码单飞”——先按 Fmin 中依赖分组 ( 无共同左部的各自成组、有共同左部的合并 )，若候选码未在任何分解关系中出现则单独加一个含候选码的关系。本题  $R_1(ABC)$  已含候选码 AB，所以无需再加候选码关系。

金句：无损连接的快速判断——分解中只要有一个关系包含候选码，就一定无损。无需画表格。

## 第 5 题

答案：

- ( 1 ) 候选码：A；最高 3NF
- ( 2 )  $R_1(BC)$ 、 $R_{21}(BE)$ 、 $R_{221}(BD)$ 、 $R_{222}(AB)$
- ( 3 ) 全部满足 BCNF

解析：这题考的是分解为 BCNF 的迭代分解流程。

### (1) 求候选码

$F = \{A \rightarrow B, B \rightarrow C, C \rightarrow D, BC \rightarrow E\}$

- A：A  $\rightarrow$  B 左侧，无右侧  $\rightarrow$  L
- B：A  $\rightarrow$  B 右侧、B  $\rightarrow$  C 左侧、BC  $\rightarrow$  E 左侧  $\rightarrow$  LR
- C：B  $\rightarrow$  C 右侧、C  $\rightarrow$  D 左侧、BC  $\rightarrow$  E 左侧  $\rightarrow$  LR



- $D : C \rightarrow D$  右侧  $\rightarrow R$
- $E : BC \rightarrow E$  右侧  $\rightarrow R$

L 类：A 必在候选码中。

- $A^+ = \{A, B, C, D, E\}$  (  $A \rightarrow B$  加 B ;  $B \rightarrow C$  加 C ;  $C \rightarrow D$  加 D ;  $BC \rightarrow E$  加 E )  $\rightarrow A^+ = U$  【正确】

候选码：A。最高范式判断：

- 1NF 【正确】 ( 原子项 )
- 2NF 检查：候选码 A 是单个属性，不存在“非主属性对码的真子集的部分函数依赖”  $\rightarrow$  2NF 【正确】
- 3NF 检查：检查非主属性对码的传递依赖。A 是单个属性，无传递路径可能  $\rightarrow$  3NF 【正确】
- BCNF 检查：每个决定因素都含码吗？
- $A \rightarrow B$  左侧 A 是候选码 【正确】
- $B \rightarrow C$  左侧 B 不是候选码 【错误】
- $C \rightarrow D$  左侧 C 不是候选码 【错误】
- $BC \rightarrow E$  左侧 BC 不是候选码 【错误】
- 存在违反 BCNF 的依赖  $\rightarrow$  BCNF 【错误】
- 最高范式为 3NF。

(2) 迭代分解为 BCNF

选违反 BCNF 的依赖进行分解。关键思路：BCNF 分解必须“迭代找违反”——每拆一次，新产生的关系必须重新计算 F 投影，然后继续检查是否所有决定因素都含码，直到每个分解关系都满足 BCNF 为止。

Step 1：选  $B \rightarrow C$  违反 ( 左侧 B 不是码 )。

- 按规则：违反依赖  $X \rightarrow Y$ ，分解为  $R_1(XY) = R_1(BC)$  和  $R_2(U - Y) = R_2(ABDE)$
- $R_1(BC)$ ：  $F_1 = \{B \rightarrow C\}$  (  $B \rightarrow C$  在  $R_1$  中保留 )  $\rightarrow$  候选码 B 是码  $\rightarrow$  BCNF 【正确】
- $R_2(ABDE)$  的 F 投影 ( 需要计算原 F 中每条 FD 的左部在  $R_2$  属性集上的闭包 )：
- $A \rightarrow B$ ：A、B 都在  $R_2$  中  $\rightarrow$  保留  $A \rightarrow B$
- $A \rightarrow D$ ：  $A^+_F = ABCDE$ ，投影到  $R_2$  得  $ABDE \rightarrow A \rightarrow D$  保留 (  $A^+_F \cap R_2$  含 D )
- $A \rightarrow E$ ：  $A^+_F = ABCDE$ ，投影到  $R_2$  得  $ABDE \rightarrow A \rightarrow E$  保留
- $B \rightarrow C$ ：C 不在  $R_2 \rightarrow$  失效
- $B \rightarrow D$ ：  $B^+_F = BCDE$ ，投影到  $R_2$  得  $BDE \rightarrow B \rightarrow D$  保留
- $B \rightarrow E$ ：  $B^+_F = BCDE$ ，投影到  $R_2$  得  $BDE \rightarrow B \rightarrow E$  保留
- $C \rightarrow D$ ：C 不在  $R_2 \rightarrow$  失效
- $BC \rightarrow E$ ：C 不在  $R_2 \rightarrow$  失效
- $F_2 = \{A \rightarrow B, A \rightarrow D, A \rightarrow E, B \rightarrow D, B \rightarrow E\}$

Step 2：检查  $R_2(ABDE)$ 。

- $R_2$  候选码：  $A^+_F = \{A, B, D, E\}$  (  $A \rightarrow B$  加 B ;  $A \rightarrow D$  加 D ;  $A \rightarrow E$  加 E ;  $B \rightarrow E$  已含 E )  $\rightarrow ABDE$  【正确】  $\rightarrow$  候选码是 A

- 违反 BCNF 的依赖：  $B \rightarrow D$  ( B 不是候选码 A，左侧 B 不含码 )；  $B \rightarrow E$  同理违反

继续分解  $R_2(ABDE)$ ，选  $B \rightarrow E$  违反。

- 按规则：分解为  $R_{21}(BE)$  和  $R_{22}(ABDE - \{E\}) = R_{22}(ABD)$
- $R_{21}(BE)$ ：  $F = \{B \rightarrow E\}$ ，候选码 B  $\rightarrow$  BCNF 【正确】
- $R_{22}(ABD)$  的 F 投影：
- $A \rightarrow B$ ：A、B 都在  $R_{22}$  中  $\rightarrow$  保留  $A \rightarrow B$
- $A \rightarrow D$ ：  $A^+_F = ABCDE$ ，投影到 ABD 得  $ABD \rightarrow A \rightarrow D$  保留
- $B \rightarrow D$ ：  $B^+_F = BCDE$ ，投影到 ABD 得  $BD \rightarrow B \rightarrow D$  保留
- $F = \{A \rightarrow B, A \rightarrow D, B \rightarrow D\}$

Step 3：检查  $R_{22}(ABD)$ 。

- $R_{22}$  候选码：  $A^+ = \{A, B, D\}$  (  $A \rightarrow B$  加 B ;  $A \rightarrow D$  加 D ;  $B \rightarrow D$  已含 D )  $\rightarrow ABD$  【正确】  $\rightarrow$  候选码是 A
- 违反 BCNF 的依赖：  $B \rightarrow D$  ( B 不是候选码 A，左侧 B 不含码 )  $\rightarrow R_{22}(ABD)$  不满足 BCNF，必须继续分解！



继续分解  $R_{22}(ABD)$ ，选  $B \rightarrow D$  违反。

- 按规则：分解为  $R_{221}(BD)$  和  $R_{222}(ABD - \{D\}) = R_{222}(AB)$
- $R_{221}(BD)$ ：F =  $\{B \rightarrow D\}$ ，候选码  $B \rightarrow BCNF$  【正确】
- $R_{222}(AB)$ ：F =  $\{A \rightarrow B\}$ ，候选码  $A \rightarrow BCNF$  【正确】

最终分解： $R = \{R_1(BC), R_{21}(BE), R_{221}(BD), R_{222}(AB)\}$

(3) 验证 BCNF

| 关系            | F 投影              | 候选码 | BCNF 验证   |
|---------------|-------------------|-----|-----------|
| $R_1(BC)$     | $B \rightarrow C$ | B   | B 是码 【正确】 |
| $R_{21}(BE)$  | $B \rightarrow E$ | B   | B 是码 【正确】 |
| $R_{221}(BD)$ | $B \rightarrow D$ | B   | B 是码 【正确】 |
| $R_{222}(AB)$ | $A \rightarrow B$ | A   | A 是码 【正确】 |

全部满足 BCNF。

金句：BCNF 分解是“迭代找违反”——每次挑一个左侧不含码的依赖  $X \rightarrow Y$  拆成  $R_1(XY)$  和  $R_2(U-Y)$ ，然后必须对新产生的每个关系重新投影 F、继续检查，直到全部满足 BCNF 为止。容易犯错的是：以为拆一次就够了，结果新关系里还藏着违反 BCNF 的依赖。

金句：BCNF 分解可能丢依赖——本题分解为 BCNF 后， $C \rightarrow D$ 、 $BC \rightarrow E$  这些原 F 中的依赖消失了（在某些分解关系中无法检查），但 BCNF 不要求保持依赖，只要求每个分解关系满足 BCNF。

陷阱警示：BCNF 分解时 F 投影的计算不是简单的“属性在则保留”。正确做法：对原 F 中每条 FD  $X \rightarrow Y$ ，求  $X^+_F$ （原 F 下的闭包），再取与当前关系属性集的交集，闭包中含目标属性才保留。本题 Step 1 中  $A \rightarrow D$ 、 $A \rightarrow E$ 、 $B \rightarrow D$  都是通过闭包投影而非直接属性匹配得到的，用  $X^+_F \cap R_2$  方法验证无误。

## 板块三：关系代数 + SQL

关系模式：图书馆场景（读者/图书/借阅）

三表： $R$ (读者号, 姓名, 类别, 注册日期) /  $B$ (图书号, 书名, 类别, 单价) /  $L$ (读者号, 图书号, 借阅日期, 归还日期)

关键点：类别在  $R$  表示身份、在  $B$  表示主题；归还日期允许 NULL。

### 第 6 题

答案：

- (1)  $\pi$  姓名 ( $R \bowtie (\pi$  读者号, 类别 ( $L \bowtie B) \div \pi$  类别 ( $B$ )))
- (2)  $\pi$  姓名 ( $R \bowtie (\pi$  读者号 ( $L \bowtie \sigma$  类别='计算机' ( $B$ )))
- (3)  $\pi$  姓名 ( $R \bowtie (\sigma$  借阅数  $\geq 3$  ( $\text{读者号 } G \text{ COUNT(DISTINCT 图书号)} \rightarrow \text{借阅数 } (L)$ ))) (注：严格应加 DISTINCT 去重，本卷接受此写法)

解析：

(1) 借过所有类别图书的读者姓名——[识别量词“所有”→选除运算 ( $\div$ ) →构造除数被除]：

- 标志词“所有类别”= 全部类别，是除运算的典型场景。
- 思路：先得到“每个读者借过的图书类别”集合，与“图书的全集类别”做除法，剩下的就是“借过所有类别的读者号”。
- Step 1 构造被除数：借阅表与图书表按图书号连接，得到“读者-类别”对—— $\pi$  读者号, 类别 ( $L \bowtie B$ )。这一步必须把类别从图书表“搬”到借阅表上，否则除法的属性对不上。
- Step 2 构造除数：图书的所有类别—— $\pi$  类别 ( $B$ )。
- Step 3 做除法：得到借过所有类别的读者号—— $\pi$  读者号, 类别 ( $L \bowtie B$ )  $\div$   $\pi$  类别 ( $B$ )。





- Step 4 连接读者表拿姓名：最终—— $\pi$  姓名 ( $R \bowtie (\pi \text{ 读者号, 类别 } (L \bowtie B) \div \pi \text{ 类别 } (B)))$ 。
- 陷阱：若直接写  $\pi \text{ 读者号 } (L) \div \pi \text{ 类别 } (B)$ ，被除数缺少类别属性，类型不匹配，除法无意义。必须先与 B 连接把"类别"挂上去。

金句："全部/所有"用除，被除数必须带除数属性——除法本质是"含有除数全集合的行"。

(2) 至少借过一本"计算机"类图书的读者姓名——[识别量词"至少有一本"→连接+选择即可]：

- 标志词"至少有一本"是存在性条件 ( $\exists$ )，不需要除法，用普通的"先选后连"组合。
- Step 1 选计算机类图书—— $\sigma \text{ 类别='计算机' } (B)$ 。
- Step 2 与借阅表连接得到"借过计算机书的借阅记录"—— $L \bowtie \sigma \text{ 类别='计算机' } (B)$ 。
- Step 3 投影出读者号—— $\pi \text{ 读者号 } (L \bowtie \sigma \text{ 类别='计算机' } (B))$ 。
- Step 4 连接读者表拿姓名—— $\pi \text{ 姓名 } (R \bowtie (\pi \text{ 读者号 } (L \bowtie \sigma \text{ 类别='计算机' } (B))))$ 。
- 陷阱：不能用除法 ( $\div$ ) 做这件事。"至少有一个"是  $\exists$ ，"全部/所有"才是  $\forall$ 。 $\forall$  转  $\exists$  转  $\div$  才是"全部"的套路；直接"存在"只需要  $\sigma \bowtie$ 。
- 陷阱：连接条件要明确——L 与 B 用图书号连接，若忘记  $\bowtie$  的公共属性，连接会变笛卡尔积，结果错误。

金句："至少一个"是存在 ( $\exists$ )，用  $\sigma \bowtie$ ；"全部/所有"是全称 ( $\forall$ )，用  $\div$ ——量词决定运算。

(3) 借了至少 3 本图书的读者姓名——[识别量词"至少 N 个"→聚集分组+计数+选择]：

- 标志词"至少 3 本"是"按读者号分组后计数  $\geq 3$ "的典型聚集场景。
- Step 1 按读者号分组并统计借书总数——读者号  $G \text{ COUNT(图书号)} \rightarrow$  借阅数 (L)。
- Step 2 筛选借阅数  $\geq 3$  的读者号—— $\sigma \text{ 借阅数} \geq 3$  (结果)。
- Step 3 连接读者表拿姓名—— $\pi \text{ 姓名 } (R \bowtie (\sigma \text{ 借阅数} \geq 3 (\text{读者号 } G \text{ COUNT(图书号)} \rightarrow \text{借阅数 } (L))))$

。

- 陷阱：必须先聚集再选择 ( $\sigma$  套在 G 外面)，不能反过来——关系代数中 G 之后不能再做普通  $\sigma$ ，只能对聚集结果做条件筛选。若写  $\sigma G \text{ COUNT(图书号)} \geq 3 (L)$  是不合法的。

- 陷阱：题目要求"按图书号去重"，但借阅表 (读者号, 图书号, 借阅日期) 中同一读者对同一图书号的不同借阅日期算多条记录。COUNT(图书号) 默认不去重 (即便属性非空)。严格意义上应写  $G \text{ COUNT(DISTINCT 图书号)}$ ，但关系代数课本常省略 DISTINCT 写法，本题接受 COUNT(图书号) 作为标准答案。阅卷以"按学号分组统计"为识别重点即可。

金句："至少 N 个"用聚集 G，先分组再选—— $\sigma$  套在 G 外面，不能套在里面。

## 第 7 题

答案：

(1) 嵌套子查询 (NOT EXISTS 双层)：

```
SELECT 姓名 FROM R
WHERE NOT EXISTS (
    SELECT 1 FROM B
    WHERE NOT EXISTS (
        SELECT 1 FROM L
        WHERE L.读者号 = R.读者号 AND L.图书号 = B.图书号
    )
);
```

(2) GROUP BY + HAVING：





```
SELECT R.姓名, COUNT(DISTINCT B.类别) AS 类别数
FROM R JOIN L ON R.读者号 = L.读者号
      JOIN B ON L.图书号 = B.图书号
GROUP BY R.读者号, R.姓名
HAVING COUNT(DISTINCT B.类别) >= 3;
```

### (3) 外连接 + 空值处理 ( LEFT JOIN + COALESCE ) :

```
SELECT R.读者号, R.姓名,
       COALESCE(SUM(B.单价), 0) AS 累计价值
FROM R
LEFT JOIN L ON R.读者号 = L.读者号
LEFT JOIN B ON L.图书号 = B.图书号
GROUP BY R.读者号, R.姓名;
```

解析：

(1) 嵌套子查询——“借过所有类别图书”——[识别需求“所有”→选 NOT EXISTS 双层→写 SQL]：

- 思路：SQL 没有直接对应关系代数  $\div$  的运算符，但可以用双重 NOT EXISTS 模拟“所有”——“借过所有类别”等价于“不存在某本某类别的书他没借过”。
- 外层 NOT EXISTS 对“图书类别”遍历：只要 B 表里有一本某类别的书 R 没借过，则该读者被排除。
- 内层 NOT EXISTS 对“借阅记录”检查：当前 R 中的读者与 B 中的书之间没有借阅记录。
- 关键子句作用：R.读者号 = L.读者号 AND L.图书号 = B.图书号 是相关条件，把外层读者和内层书关联起来。
- 陷阱：单层 NOT EXISTS 写不出“所有”——单层只能表达“不存在某读者没借过的所有书的逆”这种混乱语义。必须双层，外层“否定类别”、内层“否定借阅”。
- 陷阱：内层引用外层的 R.读者号 和 B.图书号，必须用相关子查询 ( correlated subquery )，不能写成独立的子查询。
- 陷阱：用 IN 写“借过所有”的反向：通常用 NOT EXISTS 比 NOT IN 在 NULL 时更安全——NOT IN (子查询) 在子查询返回 NULL 时结果为 UNKNOWN，整条记录被排除而非包含。NOT EXISTS 不受此影响。这题没有 NULL 干扰，但养成用 NOT EXISTS 的习惯更稳。
- 替代写法 (IN/EXISTS)：也可用 IN 实现“借过某类别”但“所有类别”必须用 NOT EXISTS。
- 金句：“所有”→双层 NOT EXISTS，外层否类别、内层否借阅；NOT EXISTS 比 NOT IN 在 NULL 时更安全。

(2) GROUP BY + HAVING——“至少 3 本不同类别图书”——[识别需求“不同类别 + 数量”→选 DISTINCT+GROUP BY+HAVING→写 SQL]：

- 思路：先关联 R/L/B 拿到每位读者借过的图书类别，再按读者分组统计“不同类别数”，用 HAVING 过滤  $\geq 3$ 。
- JOIN 链把三表连起来：R JOIN L ON 读者号 JOIN B ON 图书号。必须三表都连，否则拿不到 B.类别。
- GROUP BY R.读者号, R.姓名：SQL 标准要求 SELECT 中非聚集列都出现在 GROUP BY 中，所以 R.姓名 必须放进 GROUP BY (MySQL 早期可以省，但标准 SQL 不行)。
- COUNT(DISTINCT B.类别)：必须用 DISTINCT！否则同一读者借 5 本计算机类书，类别数会被算成 5。
- HAVING COUNT(DISTINCT B.类别) >= 3：HAVING 是对分组的条件筛选，发生在 GROUP BY 之后；WHERE 是分组前的行筛选。
- 陷阱：WHERE 与 HAVING 的区别——WHERE 不能用聚集函数 (COUNT/SUM/AVG/MAX/MIN)，HAVING 才能。若写 WHERE COUNT(DISTINCT B.类别) >= 3 报错。
- 陷阱：题目要求“不同类别”——漏 DISTINCT 是高频扣分点，写成 COUNT(B.类别) >= 3 实际筛的



是"至少借 3 本书"而非"至少 3 个不同类别"。

金句：WHERE 筛行（在 GROUP BY 之前），HAVING 筛组（在 GROUP BY 之后）；"不同 X"必须 COUNT(DISTINCT X)。

(3) 外连接 + 空值处理——"每位读者累计借阅价值，未借过的显示 0"——[识别需求"所有读者都列出 + 0 元兜底"→选 LEFT JOIN+COALESCE→写 SQL]：

• 思路：用 LEFT JOIN 保留所有读者（包括没借过书的），再用 COALESCE 把 SUM 在 LEFT JOIN 后为 NULL 的情况兜底为 0。

• FROM R LEFT JOIN L：以 R 为基准（驱动表），保留所有读者。

• LEFT JOIN B ON L.图书号 = B.图书号：再把借阅记录连到图书表拿单价。

• COALESCE(SUM(B.单价), 0)：SUM 在没有匹配行时返回 NULL，COALESCE 把它兜底为 0。

• 陷阱 1：若用 INNER JOIN，没借过书的读者会被直接过滤掉，输出缺少"未借过书"的读者——不满足"每位读者都列出"的需求。

• 陷阱 2：若不用 COALESCE，未借过书的读者累计价值会显示 NULL 而非 0——LEFT JOIN 后 SUM 是 NULL，必须显式处理。

• 陷阱 3：聚合时机——SUM 必须在 LEFT JOIN 之后做（GROUP BY 之前），不能用 SUM(B.单价) AS 累计价值 写在 SELECT 里然后再加 COALESCE 包裹整个 SUM——COALESCE(SUM(...), 0) 是正确写法。COALESCE 包裹整列聚合是标准模式。

• 陷阱 4（聚合与左连接的执行顺序）：LEFT JOIN 后再聚合时，聚合会先按 GROUP BY 分组；未借过书的读者对应的 SUM 输入全是 NULL，SUM 忽略 NULL 返回 NULL（不是 0！），所以必须 COALESCE。

• 陷阱 5（IN/EXISTS 与 NULL 的微妙差异）：用 WHERE R.读者号 IN (SELECT 读者号 FROM L) 查"借过书的读者"在 L.读者号 有 NULL 时会判为 UNKNOWN，整条记录被排除；而 WHERE EXISTS (SELECT 1 FROM L WHERE L.读者号 = R.读者号) 不受 NULL 影响。这题不涉及，但相关子查询的安全性值得留意。

金句：LEFT JOIN 保留悬浮元组 → SUM 遇 NULL 返 NULL → COALESCE 兜底 0；想"所有 X 都列出"必用外连接。

## 板块四：ER 图 + 数据库设计

### 第 8 题

答案：

(1) E-R 图（文字描述）：

• 实体 4 个：科室（矩形）、医生（矩形）、患者（矩形）、病历（矩形）

• 实体属性：科室(科室编号,科室名称,位置)；医生(医生编号,姓名,职称,专长)；患者(患者编号,姓名,性别,出生日期)；病历(病历号,建档日期,过敏史)

• 联系 3 个：

• 科室-医生：1:n 联系（菱形），一个科室有多个医生

• 医生-患者：m:n 联系（菱形），通过"就诊"联系

• 就诊联系的属性（椭圆）：就诊日期、诊断结果、处方

• 患者-病历：1:1 联系（菱形），一个患者对应一份病历

• 符号说明：实体用矩形、属性用椭圆、联系用菱形；联系类型（1:n、m:n、1:1）标在连线上

(2) 关系模式（5 个）：

• 科室(科室编号, 科室名称, 位置)

• 医生(医生编号, 姓名, 职称, 专长, \*科室编号\*) — 外码 科室编号 引用 科室

• 患者(患者编号, 姓名, 性别, 出生日期)



- 病历(病历号, 建档日期, 过敏史, \*患者编号\*) — 外码 患者编号 引用 患者
- 就诊(医生编号, 患者编号, 就诊日期, 诊断结果, 处方) — 外码 医生编号 引用 医生、患者编号 引用 患者

患者

(3) m:n 联系"就诊"的码为(医生编号, 患者编号)双方主码组合：一位医生可接诊多个患者 → 医生编号单独不能唯一标识一条就诊；一位患者可被多个医生接诊 → 患者编号单独也不能唯一标识；只有"医生编号+患者编号"组合才能标识"某医生给某患者就诊"这一具体事实。

解析：

(1) 画 E-R 图 —— 三大关键判断：

- 实体识别：题干中"每个 X 有 Y"句式 → X 是实体。容易出错的是把"病历"漏掉（它和患者是 1:1，但病历有自己独立的属性"过敏史"，所以是独立实体而非患者属性）。
- 联系类型判断：
  - 科室-医生：题干"每位医生只属于一个科室" → 一科室对多医生 → 1:n
  - 医生-患者：题干"一位医生可接诊多位患者，一位患者可被多位医生接诊" → m:n（这是关键判断点：考生容易因"医生看患者"想当然写 1:n，但反过来"一位患者可被多位医生接诊"指向 m:n）
  - 患者-病历：题干"每个患者有且仅有一份病历" → 1:1（这是陷阱：很多考生把病历当作患者的普通属性，但病历有独立的"病历号"主码，所以是独立实体）
- 联系的属性：题干"就诊时记录就诊日期、诊断结果、处方" → 这三个是"就诊"联系的属性，而非医生或患者的属性（关键陷阱：考生容易把"处方"塞给医生或患者）

(2) E-R 图转关系模式 —— 三大规则 + 1 个陷阱：

三条转换规则（必背）：

- 1:n 联系 → 在 n 方加入 1 方主码作为外码（不独立成关系）
- m:n 联系 → 独立成新关系，码为双方主码组合
- 1:1 联系 → 任选一方加入另一方主码作为外码

应用到本题：

- 1:n（科室-医生）：在医生方加"科室编号"外码 → 医生(医生编号, 姓名, 职称, 专长, 科室编号)
- m:n（就诊）：独立成新关系 → 就诊(医生编号, 患者编号, 就诊日期, 诊断结果, 处方)
- 1:1（患者-病历）：任选一方加外码。本解选在病历方加"患者编号"外码 → 病历(病历号, 建档日期, 过敏史, 患者编号)

陷阱点（必考点）：

- "1:1 联系任选一方加外码"是一般规则，但如果 1:1 联系本身有属性，必须独立成关系（不能用外码表达）。本题中患者-病历的联系本身无属性，所以可以用外码规则；如果题中说"患者-病历的联系有'建档医生'属性"，那就必须独立成关系。
- m:n 联系必须独立成关系，不能用"在任意一方加外码"的方式处理。

(3) m:n 码为什么是组合 —— 反证法：

- 假设码是"医生编号"单独：一位医生接诊多个患者 → 多个就诊记录有相同医生编号 → 违反主码唯一性
  - 假设码是"患者编号"单独：一位患者被多个医生接诊 → 多个就诊记录有相同患者编号 → 违反主码唯一性
  - 只有 (医生编号, 患者编号) 组合：每条就诊是"某医生给某患者"的唯一事件 → 唯一标识
- 金句：1:1 任选，1:n 靠多，m:n 独立成关系；联系的属性跟联系走，别塞给实体。

## 第 9 题

答案：

(1) E-R 图（文字描述）：

- 实体 3 个：图书分类（矩形）、图书（矩形）、读者（矩形）
- 实体属性：图书分类(分类编号, 分类名, 分类描述)；图书(书号, 书名, 作者, 出版社, ISBN, 单价)；



读者(读者编号, 姓名, 性别, 电话, 所在单位, 登记日期)

- 联系 2 个 :
- 图书分类-图书 : 1:n 联系 ( 一个分类包含多本图书 )
- 读者-图书 : m:n 联系 , 通过"借阅"联系
- 借阅联系的属性 : 借阅日期、应还日期、实际归还日期、续借次数
- 弱实体说明 : 无

( 2 ) 关系模式 ( 4 个 ) :

- 图书分类(分类编号, 分类名, 分类描述)
- 图书(书号, 书名, 作者, 出版社, ISBN, 单价, \*分类编号\*) — 外码 分类编号 引用 图书分类
- 读者(读者编号, 姓名, 性别, 电话, 所在单位, 登记日期)
- 借阅(读者编号, 书号, 借阅日期, 应还日期, 实际归还日期, 续借次数) — 外码 读者编号 引用 读者、书号 引用 图书

( 3 ) 完整性约束设计 :

| 关系   | 实体完整性              | 参照完整性                                       | 用户定义完整性                                            |
|------|--------------------|---------------------------------------------|----------------------------------------------------|
| 图书分类 | 主码"分类编号"非空且唯一      | 无外码                                         | 分类名 NOT NULL                                       |
| 图书   | 主码"书号"非空且唯一        | 外码"分类编号"必须引用图书分类中存在的分类编号 ( 可空 )             | ISBN UNIQUE ; 单价 > 0 ; 书名 NOT NULL                 |
| 读者   | 主码"读者编号"非空且唯一      | 无外码                                         | 性别 IN ('男','女') ; 电话 CHECK 长度=11 ; 登记日期 ≤ 今天       |
| 借阅   | 主码"(读者编号,书号)"非空且唯一 | 外码"读者编号"必须引用读者中存在的读者编号 ; 外码"书号"必须引用图书中存在的书号 | 应还日期 ≥ 借阅日期 ; 续借次数 BETWEEN 0 AND 2 ; 实际归还日期 ≥ 借阅日期 |

( 4 ) DBMS 强制实施"应还日期 ≥ 借阅日期"的机制 :

- DDL 注册阶段 : 用户在 CREATE TABLE 时声明 CHECK 约束

```
CREATE TABLE 借阅 (
    读者编号 CHAR(8),
    书号 CHAR(10),
    借阅日期 DATE,
    应还日期 DATE,
    实际归还日期 DATE,
    续借次数 INT,
    PRIMARY KEY (读者编号, 书号),
    FOREIGN KEY (读者编号) REFERENCES 读者(读者编号),
    FOREIGN KEY (书号) REFERENCES 图书(书号),
    CONSTRAINT chk_dates CHECK (应还日期 >= 借阅日期)
);
```

或用独立 ALTER TABLE :

```
ALTER TABLE 借阅 ADD CONSTRAINT chk_dates CHECK (应还日期 >= 借阅日期);
```

- 存储阶段 : DBMS 将该约束存于数据字典 ( 系统表 ) 中的约束清单
- 检查阶段 : 每次对借阅表执行 INSERT 或 UPDATE 时 , 完整性约束子系统自动对新元组 ( 或修改后的元组 ) 执行 应还日期 >= 借阅日期 判定
- 违反处理阶段 : 若约束被违反 , DBMS 拒绝执行该 SQL 语句、回滚该操作 , 并向用户返回"完整性



约束违反"错误" (如 ORA-02290、SQLSTATE 23514) ; 未违反则正常提交

• 可选的延迟约束: 若定义为 CONSTRAINT chk\_dates CHECK (应还日期 >= 借阅日期) DEFERRABLE INITIALLY DEFERRED, 则 DBMS 在事务提交时才检查 (而非每条 SQL 后立即检查) ——适用于需要事务内临时违反、提交前修正的场景

解析:

(1) 画 E-R 图 —— 完整设计流程的起点:

- 概念结构设计的产出是 E-R 图 (第 2 步设计阶段的产物, 对应第七章"画草图")
- 识别实体: 图书、读者、图书分类 3 个
- 识别联系: 图书-分类 1:n ("一本图书属于一个图书分类"), 读者-图书 m:n ("读者借阅多本、图书被多读者借阅")
- 联系的属性: 借阅日期、应还日期等是"借阅"联系的属性 (不是读者的属性, 也不是图书的属性)

(2) E-R 图转关系模式 —— 三大规则应用:

- 1:n (图书-分类): 1:n 不独立成关系 → 在 n 方 (图书) 加 1 方主码 (分类编号) 作为外码
- m:n (读者-图书借阅): m:n 独立成关系 → 借阅(读者编号, 书号, ...)
- 主外码标注:
- 主码: 每个关系的主码用下划线或加粗
- 外码: 标注引用关系 (图书.分类编号 → 图书分类; 借阅.读者编号 → 读者; 借阅.书号 → 图书)

(3) 三大完整性约束 —— 易错点密集区:

实体完整性:

- 规则: 主码非空 (NOT NULL) 且唯一 (UNIQUE)
- 本题体现: 书号、读者编号、分类编号、(读者编号, 书号) 四个主码都非空唯一
- 易错: 把"主码属性非空"漏写 → 主码约束被破坏后, 关系失去唯一标识能力

参照完整性:

- 规则: 外码要么为空 (NULL), 要么引用主表中存在的值
- 本题体现:
- 图书.分类编号: 必须引用图书分类中存在的分类编号
- 借阅.读者编号: 必须引用读者中存在的读者编号
- 借阅.书号: 必须引用图书中存在的书号
- 易错: 外码可以为空还是必须非空?
- 图书.分类编号: 可以为空 (如尚未分类的临时书)
- 借阅.读者编号和书号: 必须非空 (借阅必须有具体的人和书)

用户定义完整性:

- 规则: 用户根据业务需求自定义的约束, 常见类型:
- CHECK 约束: CHECK(应还日期 >= 借阅日期)、CHECK(续借次数 BETWEEN 0 AND 2)、CHECK(性别 IN ('男','女'))

- UNIQUE 约束: ISBN UNIQUE (不允许两本书有同一 ISBN)

- NOT NULL: 书名 NOT NULL、分类名 NOT NULL

- DEFAULT: 登记日期 DEFAULT CURRENT\_DATE

- 本题体现: 见上表

• 易错: 把"实体完整性"和"用户定义完整性"混淆 → 主码约束是实体完整性 (DBMS 自动强制); 非主属性的约束 (如 ISBN UNIQUE、单价 > 0) 才是用户定义完整性

(4) DBMS 强制实施机制 —— 触发流程:

• 第 1 步: 约束定义阶段 —— 用户在 CREATE TABLE 时声明约束 (CHECK 子句或 CREATE TRIGGER)

- 第 2 步: 约束存储阶段 —— DBMS 将约束存储在数据字典 (系统表) 中

- 第 3 步: 约束检查阶段 —— 每次 INSERT/UPDATE 操作时, DBMS 完整性约束子系统自动检查新元组是否违反约束

- 第 4 步: 违反处理阶段 —— 违反约束的操作被 DBMS 拒绝执行 (rollback 该操作), 并返回错误





信息

陷阱（必考点）：DBMS 检查约束的时机：

- 立即约束（Immediate）：在每条 SQL 语句执行后立即检查（默认）
- 延迟约束（Deferred）：到事务结束时才检查（用 SET CONSTRAINTS DEFERRED）
- 大多数 DBMS 默认立即检查，但用户可用 SET CONSTRAINTS 切换

金句：完整性约束是数据库的“自检系统”——用户声明约束，DBMS 自动检查，违反就拒绝。

## 板块五：事务 + 恢复 + 并发

### 第 10 题

答案：

- (1) UNDO = {T3, T4}, REDO = {T2}; T1 不需要任何处理
- (2) 恢复算法 5 步：① 找最后检查点 CKPT → ② 从 CKPT 正向扫描建立 UNDO 队列 {T3, T4} 和 REDO 队列 {T2} → ③ 对 UNDO 队列反向扫描日志、用旧值恢复 → ④ 对 REDO 队列正向扫描日志、用新值重做 → ⑤ 数据库回到一致状态
- (3) DBMS 自动 UNDO T3：反向扫描 T3 的日志（“T3 WRITE C, 旧值=30”），用旧值 30 把 C 改回去；与“系统故障后 UNDO T3”操作相同（都是反向扫描+用旧值恢复），但触发场景不同——事务故障只撤销 T3 一个事务，系统故障则可能撤销多个未完成事务
- (4) 介质故障恢复：① DBA 重装最近后备副本（数据库回到转储时刻）→ ② 装入转储之后的日志 → ③ REDO 转储之后所有已提交的事务（T1、T2）→ T3、T4 因未提交无需处理；与系统故障的本质区别是必须先重装后备副本（磁盘数据已物理损坏，光靠日志救不回来），系统故障则磁盘数据未坏，不需要后备副本

解析：

这题综合考日志分析 + 检查点截断 + UNDO/REDO 区分 + 4 种故障的恢复策略。核心是按“事务是否完成”和“事务相对检查点的位置”两个维度判定每个事务的命运。

#### (1) UNDO/REDO 集合的判定

正向扫描日志（从序号 1 到 15），按“事务开始 → 操作 → 提交/未提交”四象限判定：

| 事务 | 何时开始        | 何时提交          | 故障时状态 | 处理   |
|----|-------------|---------------|-------|------|
| T1 | 序号 1（检查点前）  | 序号 7（检查点前提交）  | 已提交   | 不处理  |
| T2 | 序号 4（检查点前）  | 序号 12（检查点后提交） | 已提交   | REDO |
| T3 | 序号 8（检查点后）  | 未提交           | 未完成   | UNDO |
| T4 | 序号 13（检查点后） | 未提交           | 未完成   | UNDO |

反方向再扫一遍（从序号 15 往 1 看），验证：

- T4 在末尾还没提交 → UNDO 【正确】
- T3 在 T2 之后还没提交 → UNDO 【正确】
- T2 提交了 → 需要判断是 REDO 还是不处理
- 关键点：T2 在检查点前开始、在检查点后提交——按“检查点前完成的不处理；检查点后完成的 REDO”规则，T2 需 REDO
- 为什么 T1 不需要 REDO？T1 在检查点前已经提交且落盘（系统保证检查点时刻所有已提交事务的修改已刷到磁盘）→ 不需要 REDO
- T1 在最前面已提交且在检查点前完成 → 不处理 【正确】



双重验证结论一致：UNDO={T3, T4}，REDO={T2}，T1 不处理。

判定铁律（“检查点 4 状态”口诀）：

- 检查点前开始 + 前完成（T1）→ 不处理
- 检查点前开始 + 后完成（T2）→ REDO
- 检查点后开始 + 未完成（T3、T4）→ UNDO
- 检查点前开始 + 未完成（题中未出现，若出现也 → UNDO）

反例易错点：

• “T2 提交了所以不需要 REDO” → 错。已提交是 REDO 的前提条件，不是 REDO 的反例。REDO 是“提交了但可能没落盘”，T2 在检查点后提交，到故障点之间时间很短，确实可能没落盘。

• “T3 已经写了数据库，所以不需要 UNDO” → 错。事务写库不等于事务提交，未提交的事务即使写盘也要撤销（防止脏数据）。

## （2）引入检查点后的系统故障恢复算法

算法 5 步：

- ① 在日志中找到最后一个检查点 CKPT（本题中序号 9）
- ② 从 CKPT 开始正向扫描日志（序号 9 → 15）
- ③ 建立两个队列：
  - UNDO 队列 = {检查点之后开始、故障时未完成的事务} = {T3, T4}
  - REDO 队列 = {检查点之后开始、故障时已提交的事务} = {T2}
  - 注：T1 提交在检查点前 → 不入任何队列
- ④ 对 UNDO 队列：反向扫描日志，用“旧值”恢复
  - T3：C 从 45 改回 30（用日志中 T3 WRITE C 的旧值 30）
  - T4：D 从 90 改回 70（用日志中 T4 WRITE D 的旧值 70）
- ⑤ 对 REDO 队列：正向扫描日志，用“新值”重做
  - T2：B 从 100 改成 120（用日志中 T2 WRITE B 的新值 120）
- ⑥ 数据库恢复到一致状态

为什么 T1 不需要任何处理：

T1 在检查点前已开始并在检查点前已提交（序号 1 开始 → 序号 7 提交 → 序号 9 检查点）。检查点机制的核心保证是：检查点时刻之前所有已提交事务的修改都已经写入磁盘。因此 T1 对 A 的修改（50→80）早已落盘，磁盘上 A=80，恢复时既不需要 REDO（已经在）也不需要 UNDO（不是未完成事务）。

“为什么不用对 T1 做 REDO”的反例澄清：

- REDO 的目的是“把已提交但可能没落盘的修改再做一次”——T1 的修改在检查点时保证已落盘，落盘了就不需要再做。所以 T1 跳过。
- T2 的情况不同：T2 在检查点前开始但检查点后才提交。检查点只能保证“检查点前已提交事务落盘”，T2 提交在检查点后，所以 T2 的修改可能没落盘 → 必须 REDO。

## （3）事务故障（除零异常）的处理

处理流程（伪代码）：

- ① 反向扫描日志文件，查找该事务（T3）的所有更新操作
- ② 对每个更新操作：用日志中的“旧值”去恢复数据库
  - T3 写过 C：新值 45，旧值 30 → 把 C 改回 30
- ③ 直到找到 T3 START（序号 8），事务恢复完成





与"系统故障后 UNDO T3"的异同：

| 维度      | 事务故障         | 系统故障 (UNDO 部分)     |
|---------|--------------|--------------------|
| 触发主体    | 事务自己出错       | 系统崩溃 (断电、OS 死机)    |
| 影响范围    | 仅 1 个事务 (T3) | 所有未完成事务 (本题 T3、T4) |
| 撤销方法    | 反向扫描 + 旧值恢复  | 反向扫描 + 旧值恢复        |
| 谁触发     | DBMS 自动      | 系统重启时自动            |
| REDO 部分 | 不需要 (事务未提交)  | 需要 (系统故障有 REDO 队列) |

核心相同点：UNDO 的具体动作完全一样 (反向扫描 + 旧值恢复)。

核心不同点：范围不同——事务故障只处理 1 个事务；系统故障要批量处理多个未完成事务。

#### (4) 介质故障的恢复

恢复 3 步：

- ① DBA 手动重装最近一次后备副本 (数据库回到"转储时"的状态)  
本题中：假设最近一次后备副本是在 T1 开始前做的，则副本中 A=50, B=100, C=30, D=70
- ② 装入转储时刻到故障点之间的日志文件 (序号 1-15 的全部日志)
- ③ REDO 已提交的事务
  - REDO T1: A 从 50 改成 80 (用新值)
  - REDO T2: B 从 100 改成 120 (用新值)
  - T3、T4 未提交，不处理
  - 最终: A=80, B=120, C=30, D=70

与系统故障的本质区别：

| 维度       | 系统故障          | 介质故障                              |
|----------|---------------|-----------------------------------|
| 损坏范围     | 内存数据丢失，磁盘数据未坏 | 磁盘物理损坏，数据可能丢                      |
| 需不需要后备副本 | 不需要 (光靠日志)    | 必须重装 (光靠日志救不回来)                   |
| 恢复速度     | 快             | 慢 (要装入备份)                         |
| 谁执行      | 系统自动          | DBA 手动 + 系统配合                     |
| UNDO 部分  | 需要 (撤销未完成事务)  | 不需要 (装入的是转储时的一致状态，未完成事务的修改本来就没落盘) |

关键反例澄清：

- "介质故障也需要 UNDO 未提交事务" → 错。装入后备副本后，副本里只有"转储时已落盘的数据"，未完成事务的修改本来就没装进副本，自然不需要 UNDO。
- "系统故障也要重装后备副本" → 错。系统故障时磁盘数据是好的，重装副本反而会把"已落盘的修改"覆盖回去。

金句：恢复策略的"配对铁律"——磁盘没坏用日志，磁盘坏了先装副本再追日志；故障是事务级用 UNDO，故障是系统级用 UNDO+REDO，故障是介质级用 重装+REDO。

## 第 11 题

答案：

- (1) T2 阻塞等待——T1 对 A 加 S 锁，T2 申请 X 锁，S+X 不兼容，T2 进入等待队列
- (2) 优先图节点 {T1, T2, T3}，边 {T1→T2 (A 上 r1→w2)、T3→T1 (B 上 r3→w1)}，数据 C 上



无其他事务操作（无冲突边）。优先图无环 → 冲突可串行化。等价串行顺序：T3 → T1 → T2

- (3) T1 的 2PL 序列：

扩展阶段 (Growing) : lock-S(A) → lock-X(B)  
↓ 分界点  
收缩阶段 (Shrinking) : unlock(A) → unlock(B)

注：所有加锁在所有解锁之前完成，分界点之前无 unlock，分界点之后无 lock。

- (4) 死锁。T1 持有 A 等 T2 释放 B，T2 持有 B 等 T1 释放 A，循环等待。检测方法：等待图法（节点=事务，边=等待关系，有向图中有环即死锁）；解除：选代价最小的事务 ROLLBACK（综合考虑已修改的数据量、已执行时间、回滚后重做的成本、事务优先级等；不存在所谓“年轻者优先”的口诀）。

解析：

这题综合考封锁相容性、优先图判读、2PL 两阶段划分、活锁/死锁检测。核心是用“锁矩阵 + 冲突边 + 阶段分界”三件套把并发调度的正确性讲清楚。

### (1) X/S 锁相容矩阵应用

封锁相容矩阵（必须背）：

| T2 → T1 ↓ | X 锁 | S 锁 | -（无锁） |
|-----------|-----|-----|-------|
| X 锁       | N   | N   | Y     |
| S 锁       | N   | Y   | Y     |
| -（无锁）     | Y   | Y   | Y     |

判定过程：

- T1 执行 r1(A) 之前对 A 加 S 锁（按 2 级封锁协议）→ T1 持有 A 的 S 锁
- T2 执行 w2(A) 之前申请对 A 加 X 锁
- 查表：“T1=S、T2=X” → N（不相容）
- 结论：T2 阻塞等待（不是“报错”也不是“拒绝”——DBMS 把 T2 加入 A 的等待队列）

反例澄清：

- “T2 直接报错” → 错。DBMS 不会终止事务，而是阻塞。
  - “T2 也能加 S 锁” → 错。T2 要加的是 X 锁（写锁），与 S 锁不兼容。
  - “T2 等 T1 释放 S 锁” → 对。但要进一步说清为什么释放——按 2 级协议，S 锁读完即可释放，T1 在 r1(A) 完成后会释放 S 锁（题目中 T1 接着去读 B、写 B，所以 S 锁会较早释放）。
- 金句：X 锁排他（X+X 不行、X+S 不行、S+X 不行），S 锁共享（S+S 行、X+S 不行）——只要自己持有 X 锁，别人啥都加不了。

### (2) 优先图（Precedence Graph）画法

画图步骤：

Step 1：列出所有冲突操作对（按数据分组扫描调度 S = r1(A) w2(A) r3(B) w1(B) w3(C) c1 c2 c3）：

数据 A 上的操作：r1(A), w2(A)

- T1 读 A 在 T2 写 A 之前（r-w 冲突）→ 边 T1 → T2（T1 必须先于 T2）
- 没有其他事务对 A 的操作

数据 B 上的操作：r3(B), w1(B)

- T3 读 B 在 T1 写 B 之前（r-w 冲突）→ 边 T3 → T1（T3 必须先于 T1）
- 没有其他事务对 B 的操作

数据 C 上的操作：w3(C)

- 只有 T3 写 C，没有其他事务对 C 的操作 → 无边

Step 2：画优先图：



$T3 \longrightarrow T1 \longrightarrow T2$

边汇总：

- $T1 \rightarrow T2$  ( 来自 A 上  $r1 \rightarrow w2$  )
- $T3 \rightarrow T1$  ( 来自 B 上  $r3 \rightarrow w1$  )

Step 3：判环：

路径  $T3 \rightarrow T1 \rightarrow T2$  是单向链，无回边。无环  $\rightarrow$  冲突可串行化。

Step 4：给出等价串行顺序：

按优先图拓扑序： $T3, T1, T2$  ( $T3$  最先， $T2$  最后)。

双重验证 ( 反向核对 )：把调度 S 当作"按  $T3, T1, T2$  串行执行"，检查所有冲突操作的相对次序是否一致：

- A 上  $r1 \rightarrow w2$ ： $T1$  先、 $T2$  后 【正确】 ( 与串行  $T1 \rightarrow T2$  一致 )
- B 上  $r3 \rightarrow w1$ ： $T3$  先、 $T1$  后 【正确】 ( 与串行  $T3 \rightarrow T1$  一致 )

等价串行顺序  $T3 \rightarrow T1 \rightarrow T2$  成立。

边方向判读铁律 ( 易错 )：

"写者必须等读者"—— $T_i$  读 X 在前、 $T_j$  写 X 在后  $\rightarrow$  边  $T_i \rightarrow T_j$  ( 读者先于写者 )

>

"读者必须等写者"—— $T_i$  写 X 在前、 $T_j$  读 X 在后  $\rightarrow$  边  $T_i \rightarrow T_j$  ( 写者先于读者 )

>

"后写者必须等先写者"—— $T_i$  写 X 在前、 $T_j$  写 X 在后  $\rightarrow$  边  $T_i \rightarrow T_j$  ( 先写先于后写 )

>

核心规律："先发生的"  $\rightarrow$  "后发生的"，永远从前指向后。

反例易错：

- 把  $r1 \rightarrow w2$  错画成  $T2 \rightarrow T1$   $\rightarrow$  错。 $T1$  先读  $T2$  后写，所以  $T1$  必须在  $T2$  之前。
- 漏掉数据 C 上  $w3(C)$   $\rightarrow$  注意  $w3(C)$  单独存在但没有其他事务对 C 的操作，所以确实无冲突边。不漏边 = 把"所有可能的数据"都检查一遍 ( A、B、C 三个数据全扫了 )。

### (3) 2PL 协议的两阶段划分

2PL 的核心约束：

"分界点之前只能加锁、不能释放锁；分界点之后只能释放锁、不能申请新锁"

>

简化：所有 unlock 都在所有 lock 之后

$T1$  的 2PL 序列 ( 按调度 S 中  $T1$  的操作： $r1(A) \rightarrow w1(B)$  )：

$T1$  的 2PL 序列：

扩展阶段 ( Growing Phase )：

lock-S(A)  $\leftarrow$  读 A 前加 S 锁

lock-X(B)  $\leftarrow$  写 B 前加 X 锁

—————  $\leftarrow$  分界点 ( 最后一个 lock 之后 )

收缩阶段 ( Shrinking Phase )：

unlock(A)  $\leftarrow$  A 操作完成，释放 S 锁

unlock(B)  $\leftarrow$  B 操作完成，释放 X 锁 ( 事务结束 )

判定  $T1$  满足 2PL 的依据：

- 所有加锁 ( lock-S(A)、lock-X(B) ) 都在所有解锁 ( unlock(A)、unlock(B) ) 之前完成 【正确】
- 分界点之前只有 lock，没有 unlock 【正确】
- 分界点之后只有 unlock，没有 lock 【正确】



错误序列: lock-S(A) → unlock(A) → lock-X(B) → unlock(B)

↑ 分界点

←— 扩展    —→← 收缩    →← 扩展    →→ 收缩    →

### 2PL 的关键性质：

✓

反例易错：

- "2PL 保证不发生死锁" → 错。2PL 只保证可串行化，不保证不死锁。本题第 (4) 问就是反例。
- "三级封锁协议 = 2PL" → 错。三级封锁协议是 2PL 的特例 (三级协议强制"读前加 S、写前加 X、所有锁事务结束才释放"——这天然就是 2PL 的子集，但比 2PL 更严格)。

#### (4) 死锁的检测与解除 (陷阱子问)

T1 执行 lock-X(A), lock-X(B) : T1 持有 A, 等待 B

T2 执行 lock-X(B), lock-X(A) : T2 持有 B, 等待 A

T1 等 T2 释放 B, T2 等 T1 释放 A → 循环等待 → 死锁。

```

graph TD
    T1 -- "等待" --> T2
    T2 -- "等待" --> T1
    T1 -- "↑" --> Note
    Note --- NoteText["有向环 (死锁)"]

```

| 方法   | 做法                    | 优缺点               |
|------|-----------------------|-------------------|
| 超时法  | 事务等待超过阈值时间，判定为死锁      | 简单但可能误判（事务慢不等于死锁） |
| 等待图法 | 维护事务等待关系的有向图，有向环 = 死锁 | 准确但需维护图结构         |

### 1. 选牺牲者：从环中选一个事务 ROLLBACK

- ## 2. 释放牺牲者持有的所有锁

### 3. 牺牲者重启（或者由应用程序决定重做还是放弃）



活锁 vs 死锁区分（陷阱易错）：

| 维度     | 活锁                     | 死锁                    |
|--------|------------------------|-----------------------|
| 谁在等    | T2 一个人等                | T1 和 T2 互相等           |
| 系统是否卡住 | 系统还在动（其他事务能跑）          | 系统完全卡住（这俩事务都动不了）      |
| 本题场景   | T1 释放锁后 T3 抢走、T2 永远等不到 | T1 等 T2、T2 等 T1、循环等待  |
| 解决方法   | 先来先服务（不让插队）            | 等待图检测 + 选牺牲者 ROLLBACK |

金句：活锁是“被插队”（无限等同一资源），死锁是“互相卡住”（循环等待）——一个是“我等不到”，一个是“我们都等不到”。

## 金句汇总（按板块）

### 关系理论

- L 类是“必含种子”——候选码必含 L 类属性；R 类是“必不含”——绝不在候选码中；LR 类是“待定补集”——逐个试到闭包为 U 即可。
- 求闭包要“用过的 FD 可重复扫，直到无新元素”—— $X^+$  是 X 通过 F 能推出的所有属性的最小闭包。
- 2NF 看“非主属性的部分依赖”；3NF 看“非主属性的传递依赖”；BCNF 看“所有依赖的决定因素”。
- 3NF 分解套路是“Fmin 拆无合同 + 候选码单飞”——按 Fmin 依赖分组，候选码未在分解中出现则单独加。
- BCNF 分解套路是“迭代找违反”——每次挑一个左侧不含码的依赖拆成  $R_1(XY)$  和  $R_2(U-Y)$ ，继续对新  $R_2$  检查。
- 无损连接的快速判断——分解中只要有一个关系包含候选码，就一定无损。
- 求 Fmin 三步是“拆右、去冗、缩左”——拆右看是否多属性、去冗看自身闭包是否含右部、缩左看单属性能否由其他推出。
- BCNF 分解可能丢依赖，3NF 分解可两者兼顾——范式递进越严格，保持依赖和无损连接越难同时满足。

### 关系代数+SQL

- “全部/所有”→ 关系代数用 除 ( $\div$ )，被除数必须带除数属性；SQL 用双层 NOT EXISTS（外层否类别，内层否借阅）。
- “至少一个”→ 存在性条件 ( $\exists$ )，用 连接+选择 ( $\sigma \bowtie$ )；在 SQL 中可用 EXISTS 或 IN。
- “至少 N 个”→ 关系代数用 分组+计数 (G COUNT)，先聚集后选择；SQL 用 GROUP BY + HAVING + COUNT。
- “否定/没有”→ 关系代数用 差 ( $-$ )；SQL 用 NOT EXISTS 比 NOT IN 在 NULL 时更安全。
- IN vs EXISTS → IN 接收子查询结果集，EXISTS 检查子查询是否返回行；遇 NULL 时 IN 返 UNKNOWN，EXISTS 不受影响。
- WHERE vs HAVING → WHERE 筛行（在 GROUP BY 之前），HAVING 筛组（在 GROUP BY 之后）；WHERE 不能用聚集函数。
- 外连接 + 空值处理 → LEFT JOIN 保留悬浮元组；SUM/COUNT 遇 NULL 返 NULL；COALESCE(表达式, 默认值) 兜底。
- DISTINCT 不可省 → “不同 X 数  $\geq N$ ”必须 COUNT(DISTINCT X)，否则同一 X 多次出现会被多次计数。

### ER 图+数据库设计



1. 三大联系三大规则：1:1 任选一方加外码，1:n 把 1 方主码加入 n 方，m:n 独立成关系，码为双方主码组合。
2. 联系属性跟联系走：m:n 联系若有属性（如成绩、借阅日期），必须独立成关系，属性归联系而非任一实体。
3. 1:1 联系的陷阱：若 1:1 联系本身有属性，必须独立成关系（不能用外码表达属性）。
4. 实体完整性：主码非空（NOT NULL）且唯一（UNIQUE），DBMS 自动强制。
5. 参照完整性：外码值要么为空，要么引用主表中存在的主码值。
6. 用户定义完整性：CHECK 约束 + UNIQUE + NOT NULL + DEFAULT，由用户根据业务需求声明。
7. E-R 图三要素：实体（矩形）、属性（椭圆）、联系（菱形）；联系类型（1:n、m:n、1:1）标在连线上。
8. 设计 6 步流水线：需求分析 → 概念结构（E-R 图）→ 逻辑结构（关系模式）→ 物理结构 → 实施 → 维护。前一步错了后面全错。
9. DBMS 实施约束的流程：约束定义（DDL）→ 约束存储（数据字典）→ 约束检查（每条 SQL 后）→ 违反处理（拒绝 + 报错 + 回滚）。
10. 三大完整性的层次：实体完整性是 DBMS 自动管、参照完整性是 DBMS 关联管、用户定义完整性是 DBMS 帮用户管。

## 事务+恢复+并发

1. 恢复技术核心三件套：日志 + 检查点 + UNDO/REDO——日志是“流水账”、检查点是“分界标”、UNDO/REDO 是“修复术”。
2. 检查点 4 状态速记口诀：前开始前完成不处理、前开始后完成 REDO、后开始未完成 UNDO、前开始未完成 UNDO（4 种里 3 种要操作，1 种不操作）。
3. T1 不需要处理的核心原因：T1 在检查点前已提交——检查点保证“前完成事务已落盘”，落盘了就不需要 REDO。
4. X/S 锁相容矩阵速记：X 排他（X+X 不行、X+S 不行、S+X 不行）、S 共享（S+S 行）——加了 X 锁别人啥都加不了。
5. 优先图边方向铁律：“写者必须等读者、读者必须等写者、后写者必须等先写者”——先发生 → 后发生，永远从前指向后。
6. 可串行化判定铁律：优先图无环 → 冲突可串行化；等价串行顺序 = 优先图的拓扑序。
7. 2PL 协议两阶段铁律：“先全部加完锁再开始释放”——加锁阶段不释放锁、释放阶段不加新锁，两阶段不交错。
8. 2PL 性质的“充分非必要”：遵守 2PL → 一定可串行化（充分）；可串行化 → 不一定遵守 2PL（非必要）。
9. 2PL vs 死锁的“无关性”：2PL 保证可串行化，但不保证不死锁——本题第（4）问的 T1/T2 循环等待就是反例。
10. 活锁 vs 死锁的本质区别：活锁是“被插队”（单人等）；死锁是“互相卡”（多人互等）。
11. 4 种故障的恢复策略配对：事务故障用 UNDO（日志）、系统故障用 UNDO+REDO（日志）、介质故障用重装+REDO（后备副本+日志）、病毒故障同介质。
12. 介质故障 vs 系统故障的本质区别：磁盘没坏用日志（系统故障）、磁盘坏了先装副本（介质故障）——要不要重装后备副本是铁标志。





# 易错点汇总

## 关系理论

1. L/R/LR 划分的方向：L 类只在左侧出现（必在码中），R 类只在右侧出现（必不在码中），LR 类左右都出现（待定）。容易把“只左侧”和“只右侧”搞反。
2. 闭包计算的“循环使用”：闭包计算要循环——用  $X \rightarrow Y$  加入 Y 后，新 X 集合可能触发新的依赖，要循环到无新元素为止。容易“只扫一遍”漏算。
3. 2NF 与 BCNF 的范围区别：2NF 只看“非主属性的左侧是否含码的部分函数依赖”；BCNF 看“所有依赖的左侧是否含码”。两者范围不同，BCNF 比 2NF 更严格。
4. 传递函数依赖的桥梁必须是“非主属性”：3NF 不允许非主属性对码的传递依赖—— $A \rightarrow B \rightarrow C$  中如果 B 是候选码，则不构成“传递”（因为 B 是码，可以直接决定 C）。桥梁必须是非主属性。
5. 3NF 分解中候选码单飞的时机：只有候选码未在分解关系中“完整出现”时才需要单独加。如果某个分解关系已包含候选码所有属性，无需再加候选码关系。
6. BCNF 分解必须重新投影 F：每次分解后，新关系上的 F 投影可能与原 F 大不相同——有些依赖因属性不在新关系中而失效。容易“直接套用原 F 验证 BCNF”导致误判。
7. 最小函数依赖集不唯一：Fmin 可能存在多个等价结果（取决于 Step 2 去除冗余的顺序），但每个 Fmin 都满足“每条左侧最小、不含冗余依赖、右侧单属性”。验证时要确认  $F_{min}^+ = F^+$ ，而非要求唯一解。
8. 候选码集合不能含超集：候选码集只由极小超码组成。第 1 题中 BE 是候选码，BDE 是 BE 的超集——BDE 不是独立的候选码，不能写进候选码集合。

## 关系代数+SQL

1. 除法被除数缺属性： $\pi$  读者号 (L)  $\div$   $\pi$  类别 (B) 类型不匹配。要先与 B 连接把“类别”挂上去再做除法。
2. “至少一个”误用除法： $\exists$  和  $\forall$  是不同量词，“至少一个”是存在量词，只需  $\sigma_{+ \bowtie}$ ；除法只对应“全部/所有”这种全称量词。
3. 聚集后顺序写反：关系代数中  $\sigma$  COUNT(课程号)  $\geq 3$  (L) 不合法，必须先 G 再  $\sigma$ 。SQL 中 HAVING 必须在 GROUP BY 之后，不能用 WHERE 替代。
4. DISTINCT 漏写：统计“不同类别数”必须 COUNT(DISTINCT B.类别)，否则同一类别的多本书会被算成多次。
5. LEFT JOIN 后 NULL 未处理：LEFT JOIN 保留没借过书的读者，SUM 在全 NULL 时返回 NULL 而非 0；必须用 COALESCE(SUM(...), 0) 兜底。
6. NOT IN 与 NULL 的陷阱：WHERE col NOT IN (子查询) 当子查询返回 NULL 时，结果为 UNKNOWN 而非 TRUE，导致整行被过滤。涉及“全部”语义时优先用 NOT EXISTS。
7. WHERE 不能用聚集：WHERE COUNT(\*) > 3 报错；聚集函数必须出现在 HAVING 子句中。
8. GROUP BY 列匹配：SQL 标准中 SELECT 的非聚集列必须全部出现在 GROUP BY 中；写 SELECT R.姓名 却只 GROUP BY R.读者号 在严格模式下会报错。
9. 相关子查询的相关条件：内层引用外层列时（如 R.读者号、B.图书号），必须在外层 WHERE 中显式关联 L.读者号 = R.读者号 AND L.图书号 = B.图书号，否则是独立子查询结果错。
10. 三表连接漏写中间表：第 6 题（2）和第 7 题（2）都要走 R-L-B 三表链，漏写 B 会拿不到 B.类别，查询语义变成“借过书”而非“借过某类别的书”。
11. COUNT 的去重行为：COUNT(图书号) 默认不去重（即便属性非空），COUNT(DISTINCT 图书号) 才去重。“借了至少 3 本”按严格语义应去重，但关系代数课本常省略。
12. LEFT JOIN 后聚合的细节：聚合按 GROUP BY 维度计算；未借过书的读者其 L 和 B 侧都是 NULL，SUM 忽略 NULL 返 NULL——必须用 COALESCE 兜底。

## ER 图+数据库设计

1. 联系的属性错塞给实体：m:n 联系有“成绩/时间/数量”等属性时，必须归联系而非任一实体——这是把



"联系"误当"实体属性"的最常见错误。

2. 1:1 联系当成 1:n：题干说"A 和 B 一一对应"是 1:1（如一个患者对应一份病历），而非 1:n；如果写 1:n 就丢了"唯一对应"语义。

3. m:n 联系用外码规则处理：m:n 联系必须独立成关系，不能在任意一方加外码处理——这是 E-R 转关系模型的最大陷阱。

4. 主码遗漏外码标注：关系模式转出来后，外码必须显式标注（如 \*科室编号\*），否则被扣分——外码是参照完整性的基础。

5. 三大完整性混淆：

- 主码约束 (NOT NULL UNIQUE) = 实体完整性
- 外码约束 (参照主表) = 参照完整性
- CHECK/UNIQUE/NOT NULL (非主码属性) = 用户定义完整性
- 易把"ISBN UNIQUE"当实体完整性（错，它是用户定义完整性，因为 ISBN 不是主码）

6. 完整性约束写成"应该"而非 CHECK 子句：用户定义完整性需要写成 SQL CHECK 约束形式（如 CHECK(应还日期 >= 借阅日期)），不能只说"应还日期必须 ≥ 借阅日期"就完事。

7. DBMS 实施机制答非所问：题目问"DBMS 如何强制实施"时，答"靠用户自觉"是 0 分；必须答"靠完整性约束子系统 + 数据字典 + 触发器"等具体机制。

8. 6 步设计流程跳过步骤：综合题需要从"需求分析"开始（产出数据字典/需求规格说明书），再到 E-R 图、关系模式——不能一上来就画 E-R 图。

9. 多值属性未独立处理：题干中若某属性是多值（如"作者"有多人），应独立成关系（教材第七章"实体内的多值属性"处理方式）——本题未涉及但作为扩展考点。

10. 联系名 vs 联系动词混淆：联系名要用名词性短语（"就诊""借阅""选修"），不能用动词（如"接诊"），否则读起来像动词短语而非实体间关系。

## 事务+恢复+并发

1. UNDO 是反向扫描、REDO 是正向扫描——"撤销"是从后往前找修改改回去，"重做"是从前往后找修改再做一遍。扫描方向反了救不回来。

2. 检查点截断规则：检查点前完成的事务不处理（不是 REDO），检查点后完成的事务才 REDO。"前完成不处理"是反直觉的关键点。

3. REDO 的判定不是"提交了就 REDO"——是"检查点之后才提交的事务才 REDO"。T1 提交了但提交在检查点前 → 不处理；T2 提交了但提交在检查点后 → REDO。

4. X 锁和 S 锁的相容性容易记反——X 锁是"排他"（X+X 不行、X+S 不行），S 锁是"共享"（S+S 行）。T1 加 X 锁、T2 加 S 锁 → T2 阻塞（不是 T2 成功）。

5. 优先图边方向：T<sub>i</sub> 读/写某数据、T<sub>j</sub> 写该数据 → 边 T<sub>i</sub> → T<sub>j</sub>（写者必须等读者，T<sub>i</sub> 先 T<sub>j</sub> 后）。易错点是把"先发生"和"后发生"画反。

6. 数据 C 上 w3(C) 单独时——没有其他事务对 C 的操作，没有冲突边。不漏边 = 把所有可能的数据（A、B、C）全扫一遍，单独写不构成边。

7. 2PL 不一定避免死锁——本题第（4）问中 T1/T2 都遵守 2PL（都先加锁再加锁），但循环等待仍然可能发生。2PL 只保证可串行化，不保证不死锁。

8. 2PL 是"分阶段"不是"不释放"——2PL 允许释放锁，只是要求所有加锁在所有解锁之前完成。不释放锁不是 2PL 的要求，"不分阶段"才是违反。

9. 死锁的"循环等待"是必要条件不是充分条件——循环等待只是死锁 4 必要条件之一（还有互斥、保持并等待、不可剥夺）。但数据库语境下"循环等待"几乎就是死锁的代名词。

10. 介质故障恢复不需要 UNDO——装入后备副本后，副本里只有转储时已落盘的数据（未完成事务的修改本来就没装进副本），所以只需要 REDO 不需要 UNDO。

11. 事务故障 UNDO vs 系统故障 UNDO——操作完全一样（反向扫描+旧值恢复），只是范围不同（事务故障撤销 1 个，系统故障可能撤销多个）。别把它们当成两种操作。

12. T1 在检查点前已提交但还在 REDO 队列？——错。T1 不入任何队列。检查点保证"前完成事务已落盘"，落盘的不需要 REDO。

13. 死锁解除"年轻者优先"是错误口诀——DBMS 实际选牺牲者主要看修改量、已执行时间、重做成本



、事务优先级，与"启动先后"无直接因果。教材不存在这个口诀。

